Exploration and Realtime Mapping of an Obstacle-Prone Unknown Environment Using Swarm of Autonomous Mobile Robots

A James Singh, Deepanwita Das and Sruti Singh[a]

[a]Department of Information Technology, National Institute of Technology Durgapur, West Bengal 713 209, India, Contact: jamesastrick@gmail.com

This piece of work describes research concerning the applica tion of Swarm Robotics for mapping an unknown area with unknown obstacles. The problem of mapping an unknown environment by a swarm of mobile robots is further divided into two classic problems of Multi-Robot Systems as Multi Robot Area Partitioning Problem and Area Exploration Problem. There is no centralized control. Each robot executes same algorithm in every cycle based on the local infor mation. We solve the mapping problem in two modules. We propose a distributed algorithm for each module and also uses concepts from computational geometry. In first module we solve the area partition ing problem where we divide the unknown area among the robots and assign each robot a part of the area. In second module each robot perform exploration of the area assigned and simultaneously generate a topological graph(map) of the sub-area. Thus by combining graphs of all sub parts of the area we get a map of the area. We simulate our proposed system with Player/Stage and map is constructed using python graphics library.
Keywords : Area Partition, Robot Swarm, Multi-Robot System, Distributed Algorithm.

1. INTRODUCTION

Area partition problem is an important problem in the field of multi-robot systems. Whenever we want to use a team of robots to perform any task say sweeping, painting, exploration etc, we always need to partition the area and assign it to the robots. It's basically a task division or a task allocation problem. Our main goal is to generate a map of an unknown area with obstacles. This problem is divided into two subproblems as *area partition* and *area exploration problem*. Area partition technique being the basis of most of the application of multi robot systems, here we use concepts from computational geometry in order to solve the area partition problem. Once we are able to divide the area into non overlapped parts or subparts, the next task is to assign each of the parts or the subparts to the individual robots which will further explore the area and generate the maps simultaneously. We have assumed the environment i.e, area and the obstacles are completely unknown to the robots and the team consists of N number of robots. So we divide the area into N partitions and each partition is allocated to a robot. Each robot explore the respective partition and generate the map and finally combining all the individual map we get a complete map of the area. We have created a simulation environment representing the obstacles using Player/Stage robotic simulator. As a result of the simulation, robots are able to partition, explore and generate the map of the whole region within a finite amount of time.

Area partition problem being an important problem in the field of multi-robot systems, significant amount of contribution have been done to solve the problem. Here we survey few of them. Markus et. al. [2], used grid to perform the partition of the area, knowledge of the area is known a priori and its a centralized way as robots does not execute the area division algorithm. They have overlaid the area with a grid and each of the grid cells divide the area into small polygons. Each robot uses an allocation mechanism with which they allocate themselves a polygon. If more than one robot try to allocate the same polygon,

then they communicate among themselves and agree upon one solution. Though the allocation mechanism is a decentralized one but the partition of the polygon is centralized. Mazda et. al. [3] also used a centralized concept of grid to divide the area into grid cells for sweeping. A robot take certain action to reach to next state(grid cell), for every action a cost function is defined. So a robot try to find a policy(set of actions), so that the overall cost function is minimized. Upma et. al., [4] presents circle partitioning method to divide the area, without mentioning anything about the calculation of the center and the radius. Jing et. al., [5] have proposed an multi robot exploration algorithm with limited communication, here communication requires all robots to be present in their respective destination in the same time. With this method they converted the exploration problem of global unknown environment into multi stage exploration. They have not provided any new method of exploration. Jinhui et. al., [6] proposed a different appraoch for area partition using delaunay triangulation, but the algorithm is again a centralized one.

Most of the works studied uses centralized partitioning techniques. In this paper we propose a completely distributed algorithm using concepts from computational geometry, which every robot in the system execute and finally divide the unknown polygon into non overlapping partitions. Robots are working independently from one another creating a completely distributed environment, though the robots communicate among themselves to locate other robots however, they need not to maintain the communication through out the algorithm. This makes the algorithm more effective w.r.t centralized one. The robots complete the whole process within finite amount of time without any collision or interruptions. Robots are assumed to have minimum amount of memory for better processing of the algorithm

Rest of the section is organized as follows, section 2 describes problem definition and models of the robot. Section 3 presents the proposed algorithm. In section 4 we report our simulation results and finally section 5 presents conclusion and future works.

2. PROBLEM DEFINITION, ASSUMPTIONS AND MODELS

Our problem is to explore an unknown area with obstacles and generate a map of the area using a team of robots.

Robots are assumed to be distributed across the area with one robot visible to at least one other robot i.e, they form a visibility graph. Every robot is color coded and the information of color codes is available to every robot. They use CSMA/CA as media access protocol for Wireless Communication. It is assumed that no message is lost in the network i.e, every sent message is received by the receiver.

2.1. Characteristics and Models of robot [7]:

Identical and Homogeneous: All robots are identical and have the same computational ability.

Autonomous: Every robot work asynchronously and independently in a completely distributed manner. No central authority.

Computational Model: Here we follow the basic *Observe-Compute-Move* [8] model. A *computational cycle* is defined to be a sequence of "observe", "compute" and "move" steps. Each of the robots executes same instructions in all the computational cycles. The actions taken by a robot in *compute* and *move* steps, entirely depend on the observations made in *observe* step.

Limited Visibility and Communication: Robots have limited visibility. Each robot can view only those robots which are located within a distance d from it. So, the area of visibility for each robot can be measured by a circle with radius d and the robot is located at the center of the circle. This circle is known as the *circle of visibility* and d is known as the *visibility radius*. Communication is also limited, every robot also has a communication radius, com-

munication is based on line of sight. If a robot is visible to another robot then they can communicate. In our case *communication radius* is same as *visibility radius*.

Non-Oblivious: Every robot have some amount of memory and can retain the information gathered in the previous computational cycle.

Visibility Graph: Based on the relative positions of the robots in the region, we define a visibility graph $G = (V, E)$. The vertices in the graph correspond to the robots in the swarm. Two nodes in the graph are connected by an edge iff the corresponding robots are mutually visible as shown in Fig. 2(b).

Robots are assumed to follow the following models [7]

Full-compass: Direction and orientation of both axes are common to all the robots. Each robot has its local co-ordinate system. All the robots would assume that they occupy the position $(0, 0)$ with respect to their local co-ordinate system.

Asynchronous Model: Every robot in the system operates independent computational cycle of variable length. They do not share a common clock.

3. PROPOSED ALGORITHM

Before discussing solution to the *area partition problem*, lets first discuss the communication module of robots i.e, how robots communicate with each other. In our scenario, no robot has the coordinates of each and every robot in the system or team. And coordinates of every robots is required for finding the convex hull. Thus each robot gets this information through communication.

3.1. Communication Module

Before starting communication module lets first discuss a lemma on coordinate translation.

3.1.1. Lemma 1 : Coordinate Translation

Since every robot in the team has it's own local coordinate system, so whenever a robot receives coordinate from other robots, those data

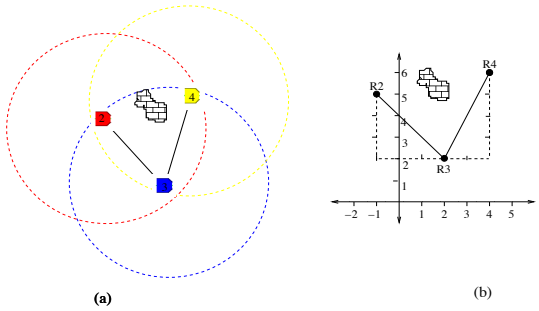

Figure 1. Coordinate Translation

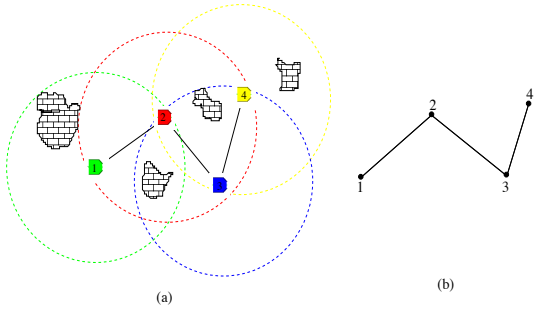

Figure 2. Robot Communication

would be w.r.t the coordinate system of the sending robot thus before using it, receiving robot should translate it into it's own coordinate systems.

In Fig. 1(a), Robot 2 is not visible to robot 4 thus cannot communicate. So robot 2 will be receiving robot 4's coordinate through robot 3. As you can see in Fig. 1(b) Coordinate of R_4 w.r.t R_3 i.e, $R_{43} = (2, 4)$ and coordinate of R_3 w.r.t R_2 i.e, $R_{32} = (3, -3)$. Now the coordinate of R_4 w.r.t R_2 i.e, R_{42} is calculated as: $R_{42} = R_{43} + R_{32}$, therefore $R_{42} = (2 + 3, 4 - 3)$, So, $R_{42} = (5, 1)$. Here, R_{ij} represent the coordinate of robot i calculated w.r.t robot j.

3.1.2. Definitions

- R : Set of Robot in the system. R $= \{R_i : i \in [1, n]\}$.
- N_{R_i} : Set of Robots that have neighbors of R_i. $N_{R_i} = \{R_j : R_j \in R\}$.
- M_{R_i} : Set having coordinates of other robots. $M_{R_i} = \{R_j < x, y >: R_j \in R\}$.
- C_{R_i} : Vector having index of all the

robots in the system along
with a flag V_j with value either 0 or 1
$C_{R_i} = \{< R_j, V_j >: R_j \in R\}$. e.g $C_{R_i} = \{< R_j, 1 >, < R_k, 0 >\}$
Here $< R_j, 1 >$ means coordinates of R_j is available to R_i
and $< R_k, 0 >$ means coordinates of R_k is not available to R_i.

- MQ_{R_i} : Message Queue of Robot R_i
  Every robot has a Message Queue which stores the raw data received from other robots, which is then translated into R_i's coordinate system and then stored into M_{R_i}.

- **Sense**(): *Observe*
  R_i sense the environment with the sensors attached and gets the coordinates of neighboring robots. Then R_i updates M_{R_i}, C_{R_i}
  Algorithm :
  Line 1. $N_{R_i} \leftarrow \{R_j | \exists j, R_j \in R\}$
  Line 2. $M_{R_i} \leftarrow \{R_j < x, y > | \forall j, R_j \in N_{R_i}\}$
  Line 3. For all j, $R_j \in M_{R_i}$
  $C_{R_i} \leftarrow < R_j, 1 >$

- **Send**(M_{R_i}) :
  R_i broadcasts the coordinates of other robots present in M_{R_i}.

- **Receive**():
  R_i Extracts data from Message Queue MQ_{R_i} and translate it to it's own coordinate system using Lemma 1 then store it in M_{R_i}.
  Line 1. $M_{R_i} \leftarrow$ **Translate**(MQ_{R_i})
  Line 2. **Return** M_{R_i}

- **Compute() :** Updates the C_{R_i}
  for all j, $R_j \in M_{R_i}$
  $V_j \leftarrow 1$
  $C_{R_i} \leftarrow < R_j, V_j >$

3.1.3. Algorithm 1: Case Study

As we can see in Fig. 2(a), we have four robots R_1, R_2, R_3 and R_4. Corresponding visibility graph of the scenario is shown in Fig.

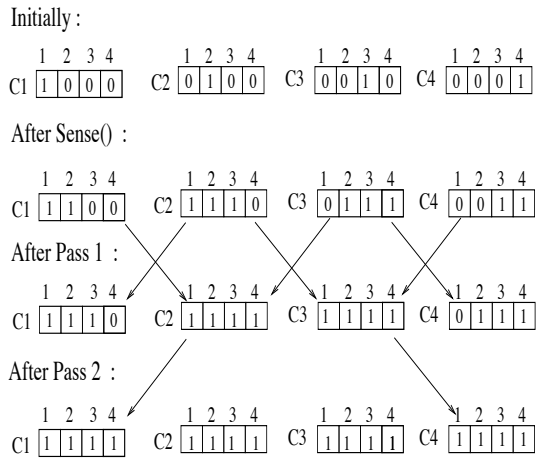

Figure 3. Execution of Communication Module

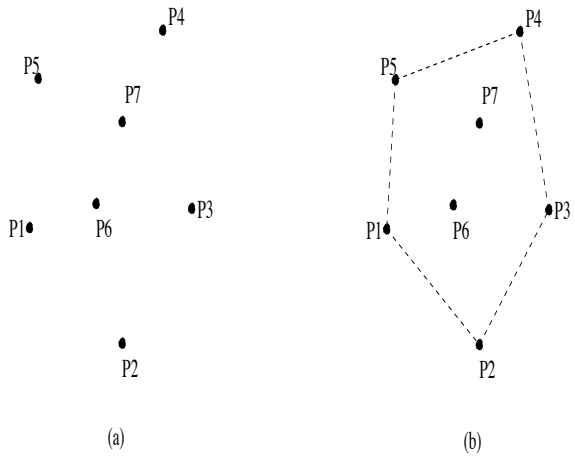

Figure 4. Convex Hull Formation

Algorithm 1 : Communication Module Executed by every robot

1: **Sense()**
2: **while** ($V_j == 1$, $\forall$ j, $V_j \in C_{R_i}$) **do**
3: **Send**(M_{R_i})
4: $M_{R_i} \leftarrow$ **Receive()**
5: **Compute()**
6: **end while**
7: **Send**(M_{R_i})

2(b). As we can see from the figure visibility is restricted. In order to get the coordinates of other robots which are not visible, every robots executes communication algorithm (**Algorithm 1**) in Table 1.
Here we execute the algorithm w.r.t robot 1, for R_1, $C_{R_1} \approx C_1$. Initially before Sense() operation content of vector C_1 is shown in figure 3. C1 = [1,0,0,0].
Line 1. After executing sense() operation, R_1 discovers one new robot, red robot(R_2). The color code information is available to all the robots. R_1 updates C1 = [1,1,0,0].
Line 2. Pass 1, R_1 executes the while loop.
Line 3. R_1 broadcasts the data i.e, M_{R_1} to the network (Send()).
Line 4. R_1 updates M_{R_1} with the data received from R_2 i.e, C2 = [1,1,1,0] coordinate information of R_3.
Line 5. R_1 updates C1 = [1,1,1,0].
Line 2. Pass 2, R_1 checks whether C1 has all the vectors = 1, which is false as Coordinate information of R_4 is not available yet, so R_1 enters the loop again.
Line 3. R_1 broadcasts the data i.e, M_{R_1} to the network (Send()).
Line 4. R_1 updates M_{R_1} with the data received from R_2 i.e, C2 = [1,1,1,1] coordinate information of R_4.
Line 5. R_1 updates C1 = [1,1,1,1].
Line 2. Now as R_1 has all the vectors = 1, so R_1 comes out of the loop.
Line 7. R_1 broadcasts the data i.e, M_{R_1} to the network (Send()).
Every other robots in the system executes the same algorithm and finally receives the coordinates of every robot in the system.

3.2. Area Partition Module

We have an unknown area for which we need to make non overlapping partitions. Say N number of robots in the system. We divide the unknown area into N parts and assign each robot a part. Before going to the algorithm of area partition, lets first discuss a lemma for finding the center of a polygon(convex).

3.2.1. Lemma 2 : Center Calculation

Let we have a polygon with n points $p1(p1_x,p1_y)$, $p2(p2_x,p2_y)$....., $pn(pn_x,pn_y)$ and C(C_x, C_y) is the center of the polygon.
Some Definitions :
d_{ij} = Distance between p_i and p_j.
$m_{(i,j)x}$ = x coordinate of p_i and p_j's mid point
$m_{(i,j)y}$ = y coordinate of p_i and p_j's mid point

$$C_x = \frac{(\sum_{i=1}^{n} m_{(i,i+1)x} * d_{i,i+1}) + m_{(n1)x} * d_{n1}}{(\sum_{i=1}^{n} d_{i,i+1}) + d_{n1}}$$

$$C_y = \frac{(\sum_{i=1}^{n} m_{(i,i+1)y} * d_{i,i+1}) + m_{(n1)y} * d_{n1}}{(\sum_{i=1}^{n} d_{i,i+1}) + d_{n1}}$$

3.2.2. Lemma 3 : Finding Convex Hull

- Convex Hull [1] : Convex Hull of a set Q of points in the Euclidean plane or Euclidean space is the smallest polygon formed by smallest set of points that contains all the points in Q. As we can see in Fig. 4(a) and 4(b).

- Algorithm for finding Convex Hull is given by Ronald Graham, Graham's Scan [9].

3.2.3. Algorithm 2: Case Study

As we see in figure 5(a) we have an unknown area with obstacles. we have a team of seven robots distributed across the area. Fig. 5(b) shows visibility graph.
Robot R_i is executing the area partition algorithm 2 in Table 3.

Table 1
Parameters

Parameters	Descriptions
R_i	Robot with id 'i'
Q	Set of Coordinates of all the robots which are obtained after Communication
V_{convex}	Set of Coordinates of robots which are the vertex of Convex Hull
V_{inside}	Set of Coordinates of robots which are inside the Convex Hull
V'_{inside}	Set of Final Coordinates of robots which were inside the Convex Hull
$insideFlag$	Flag which determines whether a robot is inside a convex hull or not
P_{left}	Nearest Coordinate of robot left to R_i
$findAdjacent_{left}$	Finds Nearest Coordinate of robot left to R_i
Eqn_{left},Eqn_{right}	Equation of lines which are used as a virtual wall
R_{ix},R_{iy}	Coordinate of Robot R_i
C_x, C_y	x and y coordinates of center of the convex hull

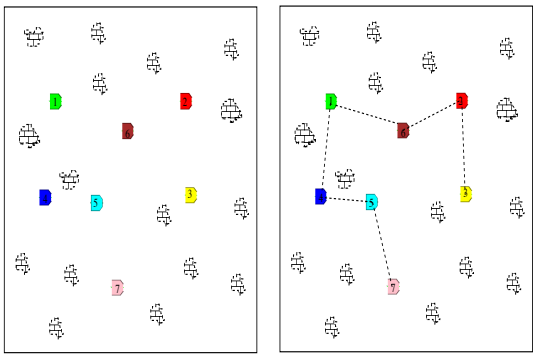

Figure 5. Unknown Area

Algorithm 2 : Area Partition

1: Q ← getCoordinates()
2: V_{convex} ← Graham's Scan(Q)
3: V_{inside} ← Q - V_{convex}
4: **if** Coordinate (0,0) ∈ V_{convex} **then**
5: insideFlag ← False
6: **else**
7: insideFlag ← True
8: **end if**
9: **if** insideFlag is True **then**
10: V'_{inside} ← Mid point of nearest side of the hull
11: Move to Mid point of nearest side of the hull
12: **end if**
13: V'_{inside} ← V'_{inside} ∪ Calculate Final point of inside robots
14: C ← Calculate Center()
15: P_{left} ← $findAdjacent_{left}(V_{convex} \cup V'_{inside})$
16: P_{rightt} ← $findAdjacent_{right}(V_{convex} \cup V'_{inside})$
17: Eqn_{left} ← $getEquation(C,\text{mid}(P_{left},R_i))$
18: Eqn_{right} ← $getEquation(C,\text{mid}(P_{right},R_i))$
19: Move to Center C
20: Turn to 180 degree
21: Execute Area Exploration Algorithm

Line 1: Coordinates of robots obtained after communication is present in Q.
Line 2-3: R_i executes Convex Hull Algorithm (Graham's Scan) on Q and stores the coordinate of robots which are vertex of convex hull in V_{convex} and coordinates of those robots which are not in convex hull set are stored in V_{inside}. Fig. 6(a) shows convex hull formed with five vertices or five robots out of those seven robots.
Line 4-7: As each robot has it's own local coordinate system, so if R_i is at the vertex of the convex hull then (0,0) must be in V_{convex}, if it belongs then insideFlag is False, which means Robot is not inside the hull, other wise insideFlag is True.
Line 8-10: If R_i is inside the convex hull, then R_i moves to the mid point of nearest

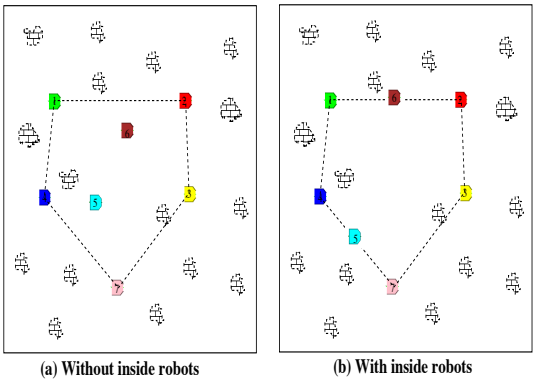

Figure 6. Convex Hull

side of the convex hull. Then R_i inserts it's new coordinate to V'_{inside}. Fig. 6(b) shows robots inside the hull moved to the mid point of nearest side of the hull.

Line 11: R_i calculates final point of all the robots inside the convex hull and stores it in V'_{inside}.

Line 12: R_i calculates center of the hull using Lemma 3.

Line 13-14: R_i finds the coordinates of nearest robots to the left and right as P_{left} and P_{right} respectively.

Line 15-16: R_i calculates equation of line which behaves as a virtual wall with points (C and P_{left}) for left side and with (C and P_{right}) for right side. Fig. 7(a) shows the virtual wall represented by the equation of the line which partition the area. Every robot has it's left wall and a right wall. In fig 7(b) every robot converge towards center and starts executing exploration of the assigned area.

3.3. Area Exploration Module

Once the area is partitioned each robot will be assigned a particular part of the area for further exploration. It is assumed that the number of parts to be explored is equal to the number of robots in the team. Each robot will be assigned to exactly one part of the area so that when each of them completes their exploration, the whole graph of the area will be generated. It is also assumed that no obstacles will further divide the assign area into two or more completely separate portions. Below mentioned are some definitions used during the algorithm.

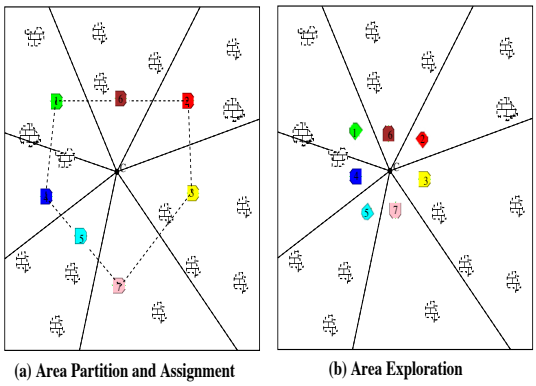

Figure 7. Partitioned Area

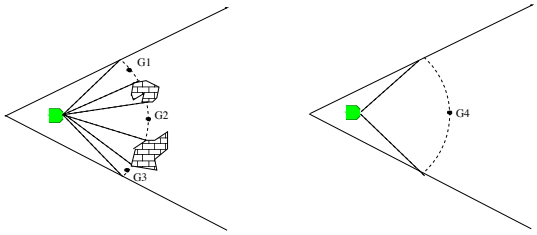

Figure 8. Representation of Gates

3.3.1. Definitions

- Point :
  Let P be the set of points in the euclidean space
  P = { P_i < x_i,y_i > : P_i ∈ Euclidean Space }

- Gate :
  Gate G_i can be defined as a mid point of a free space(through which any robot can pass) between
  (i)Two obstacles. e.g G2 in Fig. 8
  (ii)An obstacle and a boundary point. e.g G1 in Fig. 8
  (iii)Two boundary points. e.g G4 in Fig. 8
  Note : G3 is not a gate since robot cannot pass through the free space in between.
  G = { G_i < P_i > : P_i ∈ P }

Table 2. show the parameters and their corresponding descriptions used in the algorithms

Table 2
Parameters

Parameters	Descriptions
R_i	Robot
G	Set containing scanned Gates
S	Stack
V	Set containing visited Gates
G_0	Initial Gate or Starting Position
min(G)	Returns Gate from G with minimum y-coordinate (minimum x-coordinate in case of a tie)
WALL	Flag which determines whether the wall is detected by the robot or not

Algorithm 3 : Scan()

1: G ← ∅
2: Robot R_i scans the environment with the sensors attached
3: Discovers new Gates
4: Updates G
5: if discovers Wall then it sets flag WALL ← TRUE and G ← ∅
6: Return G

3.3.2. Algorithm 3 : Case Study

As we can see in Fig. 9(a), R_i is assigned with a part of the area. Now R_i starts executing area exploration algorithm.

Note : R_i is currently at G_0 as in Fig. 9(a).

Line 1-2 : R_i's current position G_0 is pushed into stack S. G_0 is also pushed into Visited set V.

Line 3 : R_i enters infinite loop.

Line 4 : R_i scans the environment, it discovers two new gates G_1 and G_2 as in Fig. 9(b). Both are inserted into G.

Line 5-9 : R_i checks whether newly discovered gates in G is already visited i.e, present in V. If it finds any then it deletes from G. Currently G_1 and G_2 are not in V.

Algorithm 4 : Area Exploration

1: Push(G_0,S)
2: V ← V ∪ { G_0 }
3: **while** TRUE **do**
4: G ← **Scan()**
5: **for** all G_i ∈ G **do**
6: **if** G_i ∈ V **then**
7: Remove G_i from G
8: **end if**
9: **end for**
10: **if** (Top(S)== G_0 and G== ∅) **then**
11: break;
12: **else if** (G≠ ∅ and Wall == FALSE) **then**
13: G' ← min(G)
14: Push(G',S)
15: V ← V ∪ { G' }
16: move(G')
17: **else if** (G == ∅ or Wall == TRUE) **then**
18: Pop(S)
19: G' ← Top(S)
20: move(G')
21: WALL ← FALSE
22: **end if**
23: **end while**

Line 10-11 : Condition does not satisfy, as G is not null.

Line 12-16 : Condition satisfied, R_i finds minimum y coordinate between G_1 and G_2, which is G_1. So, G_1 is pushed into Stack S, and also inserted into to V. Then R_i moves to G_1 as shown in Fig 10(a).

Line 17-22 : Condition does not satisfy, as G is not null.

Note : R_i is currently at G_1 as in Fig. 10(a).

Line 3 : R_i enters the loop again.

Line 4 : R_i scans the environment, it discovers two new gates G_{11} and G_{12} as in Fig. 10(a). Both are inserted into G.

Line 5-9 : As G_{11} and G_{12} are not visited, R_i does not enter the for loop.

Line 10 : Condition does not satisfy, as G is not null.

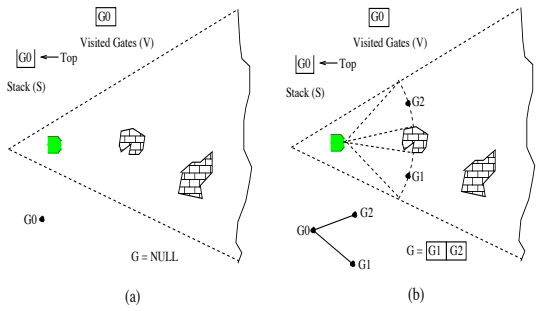

Figure 9. Step wise execution of Algorithm 3

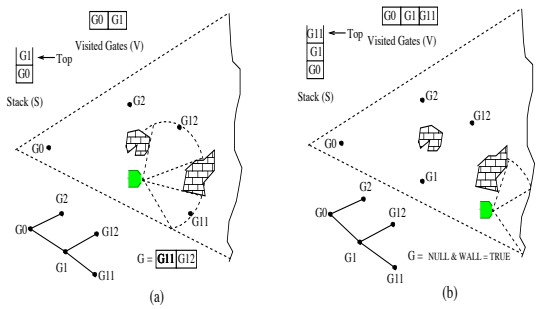

Figure 10. Step wise execution of Algorithm 3

Line 12-16 : Condition satisfied, R_i finds minimum y coordinate between G_{11} and G_{12}, which is G_{11}. So, G_{11} is pushed into Stack S, and also inserted into to V. Then R_i moves to G_{11} as shown in Fig 10(b).
Line 17-22 : Condition does not satisfy, as G is not null.
Note : R_i is currently at G_{11} as in Fig. 10(b).
Line 3 : R_i enters the loop again.
Line 4 : R_i scans the environment, now it discovers wall, so it sets WALL $\leftarrow$ True and G $\leftarrow \emptyset$.
Line 5-9 : As G $\leftarrow \emptyset$, R_i does not enter the for loop.
Line 10 : Condition does not satisfy, as Top(S) $\neq G_0$ as shown in Fig. 10(b).
Line 12-16 : Condition does not satisfied, as Wall == True.
Line 17-22 : Condition satisfied, as Wall == True. Now R_i perform Pop(S) operation. As Top(S) is G_1 after Pop(S) operation. R_i now moves to G_1 and Wall is set to False, as in Fig. 11(a).

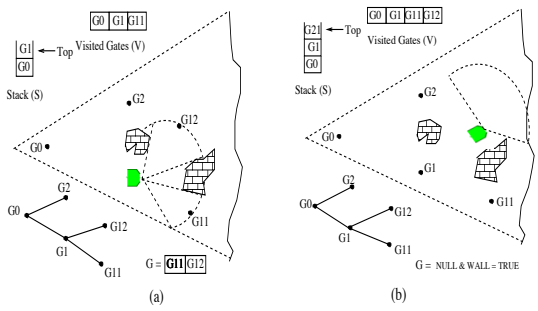

Figure 11. Step wise execution of Algorithm 3

Note : R_i is currently at G_1 as in Fig. 11(a).
Line 3 : R_i enters the loop again.
Line 4 : R_i scans the environment, it discovers two new gates G_{11} and G_{12} as in Fig. 11(a). Both are inserted into G.
Line 5-9 : As G_{11} is already is visited, R_i enters the for loop and deletes G_{11} from G
Line 10 : Condition does not satisfy, as G is not null.
Line 12-16 : Condition satisfied, R_i finds minimum y coordinate as G_{12}, as G_{12} is the only gate available. So, G_{12} is pushed into Stack S, and also inserted into to V. Then R_i moves to G_{12} as shown in Fig 11(b).
Line 17-22 : Condition does not satisfy, as G is not null.
Note : R_i is currently at G_{12} as in Fig. 11(b).
Line 3 : R_i enters the loop again.
Line 4 : R_i scans the environment, now it discovers wall, so it sets WALL $\leftarrow$ True and G $\leftarrow \emptyset$.
Line 5-9 : As G $\leftarrow \emptyset$, R_i does not enter the for loop.
Line 10 : Condition does not satisfy, as Top(S) $\neq G_0$ as shown in Fig. 11(b).
Line 12-16 : Condition does not satisfied, as Wall == True.
Line 17-22 : Condition satisfied, as Wall == True. Now R_i perform Pop(S) operation. As Top(S) is G_1 after Pop(S) operation. R_i now moves to G_1 as in Fig. 12(a) and Wall is set to False.
Note : R_i is currently at G_1 as in Fig. 12(a).
Line 3 : R_i enters the loop again.
Line 4 : R_i scans the environment, it discov-

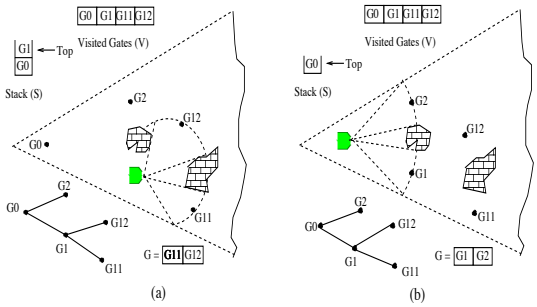

Figure 12. Step wise execution of Algorithm 3

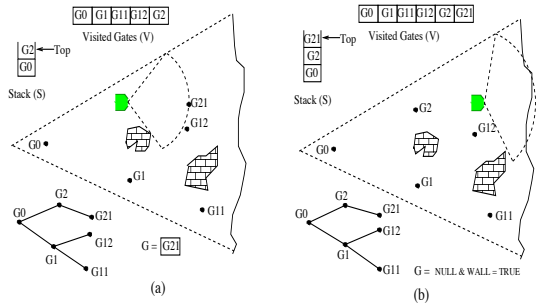

Figure 13. Step wise execution of Algorithm 3

ers two new gates G_{11} and G_{12} as in Fig. 12(a). Both are inserted into G.

Line 5-9 : As G_{11} and G_{12} are already is visited, R_i enters the for loop and deletes G_{11} and G_{12} from G. Now G $== \emptyset$

Line 10 : Condition does not satisfy, as Top(S) $\neq G_0$.

Line 12-16 : Condition does not satisfied, as $G == .$

Line 17-22 : Condition satisfied, as G is null. Now R_i perform Pop(S) operation. As Top(S) is G_0 after Pop(S) operation. R_i now moves to G_0 as in Fig. 12(b) and Wall is set to False.

Note : R_i is currently at G_0 as in Fig. 12(b).

Line 3 : R_i enters loop again.

Line 4 : R_i scans the environment, it discovers two new gates G_1 and G_2 as in Fig. 12(b). Both are inserted into G.

Line 5-9 : As G_1 is already is visited, R_i enters the for loop and deletes G_1 from G

Line 10-11 : Condition does not satisfy, as G is not null.

Line 12-16 : Condition satisfied, R_i finds gate with minimum y coordinate, which is G_2 as only G_2 is in G. So, G_2 is pushed into Stack S, and also inserted into to V. Then R_i moves to G_2 as shown in Fig 13(a).

Line 17-22 : Condition does not satisfy, as G is not null.

Note : R_i is currently at G_2 as in Fig. 13(a).

Line 3 : R_i enters loop again.

Line 4 : R_i scans the environment, it discovers a new gates G_{21} as in Fig. 13(a). G_{21} is inserted into G.

Line 5-9 : As G_{21} is not visited, So R_i does not enter the for loop.

Line 10-11 : Condition does not satisfy, as G is not null.

Line 12-16 : Condition satisfied, R_i finds gate with minimum y coordinate, which is G_{21} as only G_{21} is in G. So, G_{21} is pushed into Stack S, and also inserted into to V. Then R_i moves to G_{21} as shown in Fig 13(b).

Line 17-22 : Condition does not satisfy, as G is not null.

Note : R_i is currently at G_{21} as in Fig. 13(b).

Line 3 : R_i enters the loop again.

Line 4 : R_i scans the environment, now it discovers wall, so it sets WALL $\leftarrow$ True and G $\leftarrow \emptyset$.

Line 5-9 : As G $\leftarrow \emptyset$, R_i does not enter the for loop.

Line 10 : Condition does not satisfy, as Top(S) $\neq G_0$ as shown in Fig. 13(b).

Line 12-16 : Condition does not satisfied, as Wall == True.

Line 17-22 : Condition satisfied, as Wall == True. Now R_i perform Pop(S) operation. As Top(S) is G_2 after Pop(S) operation. R_i now moves to G_2 as in Fig. 14(a) and Wall is set to False.

Note : R_i is currently at G_2 as in Fig. 14(a).

Line 3 : R_i enters loop again.

Line 4 : R_i scans the environment, it discovers a new gates G_{21} as in Fig. 14(a). G_{21} is inserted into G.

Line 5-9 : As G_{21} is already visited, So R_i enters the for loop and deletes G_{21} from G. Now G is Null.

Line 10-11 : Condition does not satisfy, as Top(S) is not G_0.

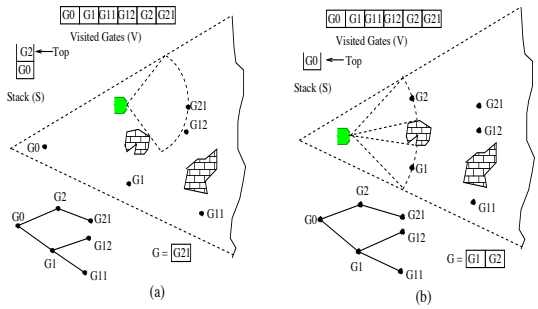

Figure 14. Step wise execution of Algorithm 3

Line 12-16 : Condition does not satisfy, G is Null
Line 17-22 : Condition satisfied, as G is null. Now R_i perform Pop(S) operation. As Top(S) is G_0 after Pop(S) operation. R_i now moves to G_0 as in Fig. 14(b) and Wall is set to False.
Note : R_i is currently at G_0 as in Fig. 14(b).
Line 3 : R_i enters loop again.
Line 4 : R_i scans the environment, it discovers two new gates G_1 and G_2 as in Fig. 12(b). Both are inserted into G.
Line 5-9 : As G_1 and G_2 are already is visited, R_i enters the for loop and deletes G_1 and G_2 from G. Now G is Null.
Line 10-11 : Condition satisfied, as G is null and Top(S) is G_0. So R_i breaks the infinite loop.
Thus finishes the Area Exploration.

4. SIMULATION RESULTS

The simulation is conducted using the Player/Stage [10] multi-robot simulation software. The Player (Version 3.2.2) and Stage (version 3.0.2) softwares is configured on Ubuntu 12.04 LTS with support of Intel Corei3 Processor with 3.00 GHz speed and 2.00 GB RAM. The robots used in this simulation are of eight different colors and having (1unit x 1unit x 1unit) dimension. The robots are equipped with the following devices:

- Infrared Laser sensors: Attached at the left and right side of the robot with sensing range upto 10 units, 180 scan lines and 180 field of view and 180 samples.

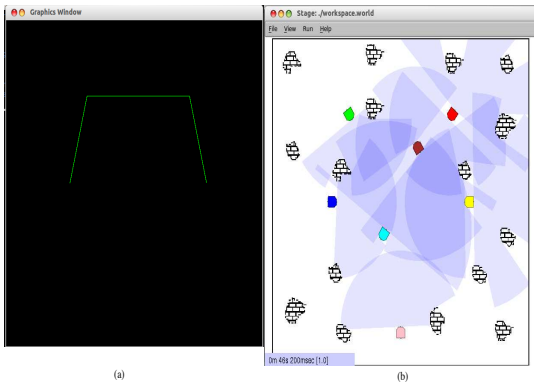

Figure 15. Formation of Convex Hull

- The Blobfinders: Attached at the left and right side of the robot and capable to recognize all the eight colors image of size 160 x 120 square unit. It has a sensing range of 10 unit and 180 field of view.

The proposed algorithm is implemented in C++ programming language, using the library libplayerc++ to communicate with the Stage. The display Graphic Window is implemented in python programming language using the library graphic.py [11] which simply reads data file of every robot and generate the graphics.

In Fig 15 convex hull is starting form and in Fig 16 convex hull is formed and non-overlapping partitions are created, robots have gathered towards center of the hull and ready for *area exploration.*
In Fig. 17 robots start exploration of assigned area, In Fig 18 yellow robot detects the obstacle and generate its location in map. Similarly for other robots. Fig 19 shows the final map generated.

Exploration generate two things, it generate the map of the unknown area and the path followed by every robot as a graph.

Note : In area exploration module Blue, Green, Cyan and Pink robots are not functioning properly because of a bug in the simulation software which deactivates the laser.

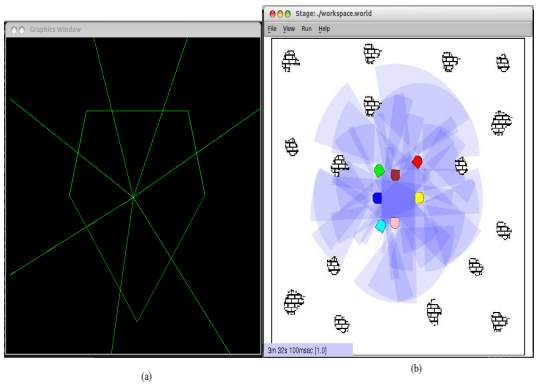

Figure 16. Convex Hull formed and Area is partitioned

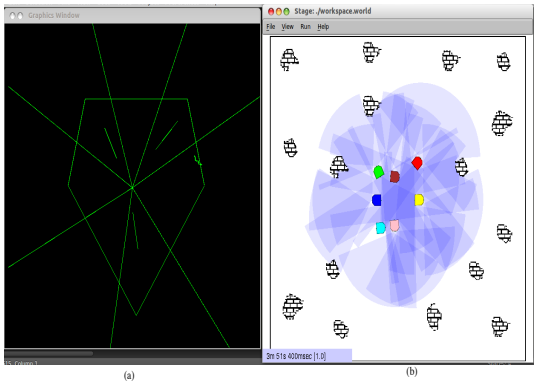

Figure 17. Exploration of assigned area of each robot

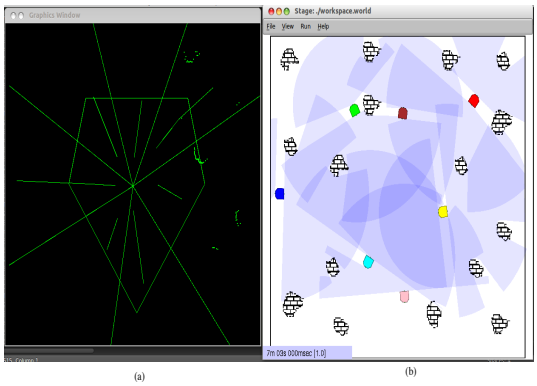

Figure 18. Exploration and generation of map

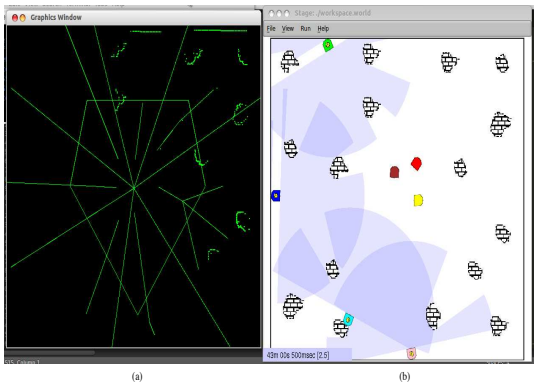

Figure 19. Completely explored and generation of map

5. CONCLUSIONS AND FUTURE WORKS

In this paper, we propose a distributed approach for division of an unknown area into non overlapping partitions. Each partition is then assigned to exactly one robot, which will explore the area and generate respective maps. Communication module is used by each robot and it results a successful identification of the coordinates of all other robots which may or may not be within the visibility range of the robot under consideration. Area partition algorithm partitions the whole area into some non overlapped parts within finite amount of time. The robots unanimously agrees upon the partition. Finally, using the area exploration algorithm, robots are able to generate the respective maps of their assigned sub area. At the end, all the maps are combined to obtain a final map of the target environment.

We have assumed that the assigned area of any robot will not be re-divided into two or more completely seperate portions due to the presence of any obstacle(s). Its a special case, we consider it as a future work which can be solved by retaining communication among the robots. We would like to address issues of fault tolerance which we have not addressed here. Also a general case where one robot completes its assigned task of generating map, it can also cooperate with any other robot for which area is still to explore. We can also think of explo-

ration of a concave polygon following several other timming and orientation models

REFERENCES

1. *http://en.wikipedia.org/wiki/Convex_hull*
2. Jager, M. and Nebel, B., "Dynamic decentralized area partitioning for cooperating cleaning robots," in *Robotics and Automation, 2002. Proceedings. ICRA '02. IEEE International Conference on (Volume:4)*, 2002, 3577 - 3582 vol.4.
3. Mazda A. and Peter S., "A Multi-Robot System for Continuous Area Sweeping Tasks," in *Learning and Adaption in Multi-Agent Systems, Springer Berlin Heidelberg*, 2006.
4. Jain, U. and Tiwari, R. and Majumdar, S. and Sharma, S., "Multi Robot Area Exploration Using Circle Partitioning Method," in *International Symposium on Robotics and Intelligent Sensors 2012 (IRIS 2012)*, Procedia Engineering 41 (2012) 383 387.
5. Jing Y. and Yalou H. and Tong T. and Fengchi S.,"A Cooperative Approach for Multi-Robot Area Exploration," in *2010 IEEE/RSJ International Conference on Intelligent Robots and Systems October 18-22, 2010, Taipei, Taiwan*
6. Jinhui Dong,"An Optimum Convex Decomposition Algorithm of Simple Polygon Based on Its Adjacent Concave Vertex Connection," in *2011 International Conference on Intelligence Science and Information Engineering, IEEE, 2011*
7. Das, D. and Mukhopadhyaya, S, "Painting an Area by Swarm of Mobile Robots with Limited Visibility," in *Wireless Networks and Computational Intelligence Communications in Computer and Information Science Volume 292* , 2012, pp 446-455.
8. Das, D. and Mukhopadhyaya, S, "An Algorithm for Painting an Area by Swarm of Mobile Robots," in *Int. Conf. on Control, Automation and Robotics, Singapore*, 2011, pp. C1C6.
9. Graham, R.L, "An Efficient Algorithm for determining the convex hull of a finite planar set," in *North-Holland Publishing Company*, 1972.
10. *http://en.wikipedia.org/wiki/Player_Project*
11. *http://mcsp.wartburg.edu/zelle/python/graphics.py*

A James Singh is currently Project Associate at Department of Computer Science, Indian Institute of Technology Kanpur, India. He obtained his Bachelor of Technology in Electronics and Communication Engineering from National Institute of Technology Silchar, India. He received his Masters degree in Information Technology from National Institute of Technology Durgapur, India with Distinction.

Deepanwita D. is currently an Assistant Professor at Department of Information Technology, National Institute of Technology Durgapur, India. She obtained her Bachelor of Technology in Information Technology from National Institute of Technology Durgapur. She received her Masters degree in Information Technology from Jadavpur University. She is also currently persuing her PhD degree at National Institute of Technology Durgapur.

Sruti K. is currently an Undergraduate Student at Department of Information Technology, National Institute of Technology Durgapur, India.